

PerfectInfo

安全分析报告

PerfecttheinformationActivity 模块

报告编号: SEC-2026-0515-002

风险评级: 高 (High)

报告日期: 2026-05-15

仅用于安全研究与教学, 禁止用于任何非法用途

由 Kiro 协助整理

PerfecttheinformationActivity 安全分析报告

报告对象:

`com.phone.secondmoveliveproject.activity.login.PerfecttheinformationActivity`

分析方法: 静态代码审计 (基于 JADX 反编译片段) **风险等级:** 高 (High) 报

告日期: 2026-05-15

一、概览

这段代码是一个 Android 应用的"完善个人信息"页面。功能包括:

1. 调用 OSS 上传头像图片
2. 选择生日 (DatePickerDialog)
3. 提交昵称、性别、邀请码、密码、生日等信息到后端 `APP_completeMaterial` 接口
4. 注册成功后写入本地 GreenDAO 数据库, 跳转到 `MainActivity`
5. 屏蔽了返回键

整体看是注册流程的最后一步。但代码中存在**多个安全风险点和工程质量问题**, 下文逐项分析。

二、风险一览表

编号	风险类型	严重程度	涉及代码位置
R-01	密码使用 MD5 单向哈希（弱算法）	严重	<code>Md5Util.toMD5(this.etPassword.getText().toString())</code>
R-02	设备指纹 OAID 上传，缺乏用户告知	高	<code>OAIDUtils.getOAID(this)</code>
R-03	调试日志泄露敏感数据	高	<code>System.out.println("-----> " + str2);</code>
R-04	邀请码未做格式校验，存在注入与刷量风险	中	<code>httpParams.put("yaoqingma", ...)</code>
R-05	<code>onKeyDown</code> 屏蔽返回键，UX/合规问题	中	<code>if (i == 4 && ...) return true;</code>
R-06	性别字段拼接字符串而非类型化传输	中	<code>httpParams.put("sex", this.sexType + "");</code>
R-07	JSON 解析无字段存在性校验，崩溃风险	中	<code>JSONObject.getInt(...)</code>
R-08	DAO 每次新建数据库连接，资源泄露	中	<code>new DaoMaster(new DbOpenHelper(...))</code>

编号	风险类型	严重程度	涉及代码位置
R-09	头像默认值硬编码 常量 URL	低	<code>QyCommon.DEFAULT_WOMAN_AVATAR</code>
R-10	<code>onActivityResult</code> 缺 resultCode 校验	低	<code>if (i == 17 && intent != null)</code>

三、风险详解

R-01 密码使用 MD5 （ 严重）

问题代码

```
if ("quickLogin".equals(this.type)) {  
    httpParams.put(BaseConstants.PWD,  
        Md5Util.toMD5(this.etPassword.getText().toString()));  
}
```

危害

- **MD5 自 2004 年起已被证明可碰撞**，2012 年后被 NIST 列为不推荐用于安全场景的算法。
- **没有加盐 (salt)**：相同密码会得到相同哈希，攻击者用彩虹表可在秒级反查常见密码。

- 即使后端再做一次哈希，**前端这一层完全是"安慰剂"**，对中间人攻击毫无防护。

复现方式

抓包后，把 `pwd=e10adc3949ba59abbe56e057f20f883e` 直接丢进彩虹表网站，即可还原出 `123456`。

修复建议

```
// 不要在客户端 hash 密码，让 HTTPS 保护明文传输
// 服务端使用 bcrypt / scrypt / argon2 加盐存储

httpParams.put(BaseConstants.PWD, etPassword.getText().toString().trim());
// 同时强制要求 HTTPS: OkHttp 增加 ConnectionSpec.MODERN_TLS
```

核心原则：密码哈希永远是服务端的责任，客户端做 hash 既不能提升安全性，还会让"忘记密码""验证旧密码"等流程更复杂。

R-02 设备指纹 OAID 上传（高）

问题代码

```
String oaid = OAIDUtils.getOAID(this);
httpParams.put("equipmentId", oaid);
((PostRequest) ... .headers("equipmentId", oaid))
```

危害

- **OAID 是中国移动安全联盟推出的设备标识符**，长度 32 位，跨应用可关联同一设备。
- 在 body 和 header 各塞一遍，强化了"反卸载追踪"。
- 中国《个人信息保护法》第 13 条要求**单独同意**才能收集设备识别信息，此处未见同意书或授权弹窗。

合规风险

- 上架华为 / 小米 / OPPO 等应用商店时会被驳回。
- 工信部 App 违法违规收集个人信息专项治理高频通报。

修复建议

1. 在隐私政策中**明确告知**采集 OAID 用途。
2. 用户**首次启动时弹出同意书**，未同意则不采集。
3. 改用**会话级随机 ID**（每次登录生成 UUID），仅用于风控。

```
String equipmentId = UserConfig.hasUserAgreedToTracking()  
    ? OAIDUtils.getOAID(this)  
    : ""; // 未同意则不上传
```


R-03 调试日志泄露敏感数据（高）

问题代码

```
public void success(String str2) {  
    System.out.println("-----> " + str2); // OSS 返回的 URL  
    PerfecttheinformationActivity.this.upWanShanData(str2);  
}  
public void failure() {  
    System.out.println("-----> failure");  
}
```

危害

- `System.out.println` 在 Android 中会输出到 **logcat**。
- Android 7.0 (API 24) 之前任意 App 可读其他 App 的日志；7.0+ 虽然限制了，但**已 root 设备 + 第三方日志收集 SDK** 仍可拿到。
- 日志里可能包含 OSS URL 中的 **签名串、AccessKey、token**。

修复建议

```
// 1. 用统一封装的 Logger，区分环境
LogUtils.d("PerfectInfo", "OSS upload success");

// 2. release 包通过 ProGuard 移除日志
// proguard-rules.pro
-assumenosideeffects class android.util.Log {
    public static *** d(...);
    public static *** v(...);
    public static *** i(...);
}
-assumenosideeffects class java.io.PrintStream {
    public *** println(...);
}
```

R-04 邀请码缺乏校验（中）

问题代码

```
httpParams.put("yaoqingma", this.et_yaoqingma.getText().toString());
```

危害

- 用户输入直接拼到 HTTP 参数中，**没有长度、字符集、空白判断**。
- 后端如果用字符串拼 SQL（很多老项目这么干），**可能 SQL 注入**。
- 邀请码若有奖励机制，可被脚本批量遍历刷量。

修复建议

```
String code = et_yaoqingma.getText().toString().trim();
if (!TextUtils.isEmpty(code)) {
    if (!code.matches("[A-Za-z0-9]{4,12}$")) {
        ToastshowUtils.showToastSafe("邀请码格式不正确");
        return;
    }
}
httpParams.put("yaoqingma", code);

// 同时在服务端增加：
// - 邀请码失败次数限制 (rate limit)
// - 同 IP / 同设备 短时间多次提交触发风控
```

R-05 屏蔽返回键（中）

问题代码

```
@Override
public boolean onKeyDown(int i, KeyEvent keyEvent) {
    if (i == 4 && keyEvent.getAction() == 0) {
        return true;
    }
    return super.onKeyDown(i, keyEvent);
}
```

危害

- **强制用户必须完成信息填写才能离开**，是典型的"暗黑模式 (Dark Pattern)"。
- 违反 Google Play 上架政策中 **"User Controls"** 条款。

- 用户无法通过返回键离开，会**直接卸载应用**，影响留存率。

修复建议

```
@Override
public boolean onKeyDown(int i, KeyEvent keyEvent) {
    if (i == KeyEvent.KEYCODE_BACK) {
        new AlertDialog.Builder(this)
            .setMessage("还没完成资料，确定要离开吗? ")
            .setPositiveButton("离开", (d, w) -> finish())
            .setNegativeButton("继续填写", null)
            .show();
        return true;
    }
    return super.onKeyDown(i, keyEvent);
}
```

R-06 性别字段拼接字符串（中）

问题代码

```
httpParams.put("sex", this.sexType + "");
httpParams.put("comeFrom", QyCommon.comeFrom + "");
```

危害

- `int + ""` 是反模式，可读性差。
- 如果 `sexType` 因初始化遗漏为 `0`，会产生**未定义性别**的脏数据。

修复建议

```
if (sexType != 1 && sexType != 2) {  
    ToastshowUtils.showToastSafe("请选择性别");  
    return;  
}  
httpParams.put("sex", String.valueOf(sexType));
```

R-07 JSON 解析无字段校验（中）

问题代码

```
JSONObject jsonObject = new JSONObject(str2);  
int i = jsonObject.getInt(C2453a.f2966j);  
String string = jsonObject.getString("msg");
```

危害

- `getInt` / `getString` 在字段缺失时会**抛出 `JSONException`**。
- 虽有 `try/catch`，但只是 `printStackTrace()`，loading 不会消失，**用户感觉应用卡死**。
- 服务端字段名变化或下发空字段时直接挂掉。

修复建议

```
try {
    JSONObject obj = new JSONObject(str2);
    int code = obj.optInt("code", -1);
    String msg = obj.optString("msg", "未知错误");
    if (code == 0) {
        // ...
    } else {
        ToastshowUtils.showToastSafe(msg);
    }
} catch (JSONException e) {
    LogUtils.e(TAG, "parse fail", e);
    ToastshowUtils.showToastSafe("数据异常，请重试");
} finally {
    hideLoading(); // ← 关键！异常路径也要隐藏 loading
}
```

R-08 DAO 每次新建数据库（中）

问题代码

```
UserDataBeanDao userDataBeanDao = new DaoMaster(
    new DbOpenHelper(PerfecttheinformationActivity.this,
        UserDataBeanDao.TABLERNAME).getWritableDatabase()
    ).newSession().getUserDataBeanDao();
```

危害

- 每次回调都 `new DbOpenHelper`，**SQLite 连接没关闭**。
- 长时间使用会出现 `SQLiteDatabaseLockedException`、`Too many open files`。
- GreenDAO 推荐**全局单例 DaoSession**。

修复建议

```
// 在 Application.onCreate 中初始化
public class App extends Application {
    private static DaoSession daoSession;
    @Override
    public void onCreate() {
        super.onCreate();
        DaoMaster.OpenHelper helper = new DbOpenHelper(this, "user.db");
        daoSession = new DaoMaster(helper.getWritableDatabase()).newSession();
    }
    public static DaoSession getDaoSession() { return daoSession; }
}

// 业务代码：
UserDataBeanDao dao = App.getDaoSession().getUserDataBeanDao();
```

R-09 头像默认值硬编码（低）

问题代码

```
httpParams.put("ossImage", QyCommon.DEFAULT_WOMAN_AVATAR);
```

修复建议

把默认头像放到**服务端配置**或 **Remote Config**（Firebase / 阿里 ACM），客户端传 `""` 由后端兜底。这样换头像不需要发版。

R-10 onActivityResult 缺 resultCode 校验（低）

问题代码

```
if (i == 17 && intent != null) {  
    List<String> listObtainPathResult = Matisse.obtainPathResult(intent);  
    ...  
}
```

危害

- 用户在 Matisse 选择器中**点取消（resultCode = RESULT_CANCELED）**也会进入这个分支吗？
- `Matisse.obtainPathResult` 在取消时返回空 List，不会崩，但会**触发空 Iterator**，浪费一次 UI 渲染。

修复建议

```
@Override  
protected void onActivityResult(int i, int i2, Intent intent) {  
    super.onActivityResult(i, i2, intent);  
    if (i != 17 || i2 != RESULT_OK || intent == null) return;  
    List<String> paths = Matisse.obtainPathResult(intent);  
    if (paths == null || paths.isEmpty()) return;  
    this.headPath = paths.get(paths.size() - 1); // 取最后一张  
    HelperGlide.loadHead(this, this.headPath, this.image_heard);  
}
```


四、综合修复优先级

优先级	任务	预估工时
P0	移除客户端 MD5，强制 HTTPS	1h
P0	release 包关闭所有 println / Log.d	0.5h
P0	OAID 采集合规化（同意书 + 隐私政策）	4h
P1	DAO 改单例，修复返回键 UX	2h
P1	JSON 解析改 <code>optInt/optString</code> ，加 finally hideLoading	1h
P2	邀请码格式校验、resultCode 检查	0.5h
P2	头像默认值上移到服务端	1h

合计：约 10 小时可完成基础整改。

五、修复后参考代码（节选）

```
// ===== 完整版 upWanShanData =====
private void upWanShanData(String ossUrl) {
    // 1. 参数装配
    HttpParams params = new HttpParams();
    params.put("ossImage", TextUtils.isEmpty(ossUrl)
        ? "" // 让服务端兜底
        : ossUrl);
    params.put("shengri", tv_birthday.getText().toString());
    params.put("nicheng", sexType == 2
        ? et_Name.getText().toString().trim()
        : "");
    params.put("yaoqingma", validateInviteCode(et_yaoqingma.getText().toString()));
    params.put("sex", String.valueOf(sexType));
    params.put("comeFrom", String.valueOf(QyCommon.comeFrom));
    params.put("definition", tvSM.getText().toString());
    params.put("appName", BuildConfig.APPLICATION_ID);

    // 2. 设备 ID (合规)
    if (PrivacyUtil.isUserAgreed()) {
        params.put("equipmentId", OAIDUtils.getOAID(this));
    }

    // 3. 不在客户端 hash 密码
    if ("quickLogin".equals(type) && sexType == 2) {
        params.put(BaseConstants.PWD, etPassword.getText().toString());
    }

    // 4. 请求
    EasyHttp.post(BaseNetworkAllApi.APP_completeMaterial)
        .params(params)
        .accessToken(true)
        .timeStamp(true)
        .execute(new SimpleCallBack<String>() {
            @Override
            public void onError(ApiException e) {
                hideLoading();
                ToastshowUtils.showToastSafe("网络异常，请重试");
            }
        })
}
```

```

        @Override
        public void onSuccess(String resp) {
            hideLoading();
            handleCompleteMaterialResp(resp);
        }
    });
}

private String validateInviteCode(String raw) {
    String code = raw == null ? "" : raw.trim();
    if (!TextUtils.isEmpty(code) && !code.matches("[A-Za-z0-9]{4,12}$")) {
        throw new IllegalArgumentException("邀请码格式不正确");
    }
    return code;
}

private void handleCompleteMaterialResp(String resp) {
    try {
        JSONObject obj = new JSONObject(resp);
        int code = obj.optInt("code", -1);
        String msg = obj.optString("msg", "未知错误");

        if (code != 0) {
            ToastshowUtils.showToastSafe(msg);
            return;
        }

        // 数据库改单例
        UserDataBeanDao dao = App.getDaoSession().getUserDataBeanDao();
        List<UserDataBean> list = dao.queryBuilder()
            .where(UserDataBeanDao.Properties.States.eq(1))
            .list();

        if (list != null && !list.isEmpty()) {
            UserDataBean bean = list.get(0);
            bean.setSex(String.valueOf(sexType));
            dao.update(bean);
        }
    }
}

```

```
MinePersonalBean userBean = UserBeanHelper.getUserBean(this);
userBean.getData().setSex(sexType);
if (list != null && !list.isEmpty()) {
    userBean.getData().setId(list.get(0).getUserId());
}
UserBeanHelper.saveUserBean(this, userBean);

startActivity(new Intent(this, MainActivity.class));
finish();

} catch (JSONException e) {
    LogUtils.e(TAG, "parse error", e);
    ToastshowUtils.showToastSafe("数据异常");
}
}
```

六、结论

这段代码反映了一个**"功能能跑、安全合规欠账"**的典型 Android 项目。

- **必须立即修复**：客户端 MD5、调试日志泄露、OAID 合规
- **影响上架与留存**：返回键屏蔽、数据库连接泄露
- **代码质量层面**：JSON 解析、参数校验、字符串拼接

整改后，该模块可达到**应用商店上架基本要求**及**等保 2.0 三级移动应用**最低安全基线。

本报告仅用于安全研究与教学目的，所有结论基于反编译片段，可能与原始项目存在偏差。请勿用于任何非法用途。 由 Kiro 协助分析整理 •
2026-05-15